

Programowanie I R

Seria 13: Matplotlib, Git

Agnieszka Makulska

8 czerwca 2026

Zadanie 1. – Rysunki

Korzystając z biblioteki `matplotlib`, stwórz rysunek złożony z trzech wykresów w niestandardowym układzie: jeden szeroki wykres na górze i dwa mniejsze na dole (użyj `GridSpec` lub `subplot_mosaic`).

- Wykres 1 (górny) – narysuj trzy przebiegi: drgania harmoniczne: $x(t) = A \sin(\omega t)$, drgania tłumione: $x(t) = Ae^{-\gamma t} \sin(\omega t)$ oraz obwiednię tłumienia: $\pm Ae^{-\gamma t}$ (linią przerywaną). Dodaj na wykresie legendę. Przyjmij $A = 1$, $\omega = 2\pi$, $\gamma = 0,4$, $t \in [0, 5]$.
- Wykres 2 (dolny lewy) – wykres fazowy drgań tłumionych: prędkość $\dot{x}(t)$ w funkcji położenia $x(t)$. Pokoloruj linię według czasu (użyj `scatter` z parametrem `c=t`).
- Wykres 3 (dolny prawy) – histogram wartości $x(t)$ dla drgań tłumionych.

Niech każdy wykres ma tytuł i podpisane osie. Dodaj jeden wspólny tytuł całego rysunku (`suptitle`). Użyj innego stylu niż domyślny (https://matplotlib.org/stable/gallery/style_sheets/style_sheets_reference.html).

Zadanie 2. – Pierwsze repozytorium Git

Git to program działający lokalnie na komputerze, który śledzi każdą zmianę w kodzie, pozwala cofnąć się do poprzednich wersji i pracować równolegle na wielu gałęziach. **GitHub** to serwis w chmurze do przechowywania repozytoriów Git. Umożliwia pracę zespołową, otwarte projekty i automatyczne testy.

Założ pierwsze repozytorium Git za pomocą poniższych komend.

```
git config --global user.name "Twoje Imie"
git config --global user.email "twoj@email"

#sklonuj istniejące repozytorium:
git clone git@github.com:AgnieszkaMakulska/Programowanie_IR_2026.git
# lub utwórz własne:
mkdir Programowanie && cd Programowanie
git init
# dodaj swoje pliki, potem:
git status # sprawdź zmiany
git add . # dodaj wszystko co znajduje się w tym folderze
git commit -m "pierwszy commit" # tworzymy commit z opisem
```

Możesz przełączać się między gałęziami (różnymi wersjami kodu) w następujący sposób:

```
git branch # wyświetla gałęzie
git checkout -b nazwa_galezi #tworzy nową gałąź i przechodzi na nią
git switch main # przechodzi na gałąź main
```

Aby Twoje repozytorium znalazło się na GitHubie, musisz je "wypchnąć" (git push). Najlepiej zacząć od ustawienia bezpiecznego połączenia między komputerem a Twoim kontem na GitHubie przez klucz ssh:

```
ssh-keygen -t ed25519 -C "twoj@email" # generujemy klucz
#kliknij Enter dla domyślnej lokalizacji (~/.ssh/id_ed25519)
eval "$(ssh-agent -s)" # uruchom agenta SSH
ssh-add ~/.ssh/id_ed25519 # dodajemy klucz do agenta
cat ~/.ssh/id_ed25519.pub # kopiujemy klucz
# Wejdź na github.com - Settings - SSH and GPG keys
# Kliknij "New SSH key", wklej zawartość id_ed25519.pub
ssh -T git@github.com # test połączenia
```

Wejdź na github.com - przycisk New, aby otworzyć nowe repozytorium na GitHubie. Jeśli chcesz wypchnąć na swoje konto sklonowane repozytorium, użyj:

```
git remote set-url origin git@github.com:twojanazwa/twojerepo.git
git push -u origin main
```

Jeśli zaczynasz nowe repozytorium od zera:

```
git remote add origin git@github.com:twojanazwa/twojerepo.git
git push -u origin main
```

Przydatne komendy:

<code>git pull</code> pobierz zmiany z GitHub	<code>git log --oneline</code> historia commitów w skrócie	<code>git diff</code> pokaż niezapisane zmiany
<code>git branch nazwa</code> utwórz nową gałąź	<code>git switch nazwa</code> przełącz gałąź	<code>git merge nazwa</code> scal gałąź z bieżącą
<code>git stash</code> schowaj niezapisane zmiany	<code>git restore plik</code> cofnij zmiany w pliku	

Opracowanie: Agnieszka Makulska